

Anish Rajeev

770-401-6112 | anishrajeev@proton.me | [LinkedIn](#) | [GitHub](#)

EDUCATION

Carnegie Mellon University | Pittsburgh, PA

May 2026

Bachelor of Science in Mathematics (Discrete Math & Logic Concentration)

GPA: 3.6

Relevant Courses: HOT Compilers, Functional Programming, Complexity Theory, Category Theory, Model Theory

EXPERIENCE

Resident | Axiom Math

June 2026 – Present

Lean, Python

- Designed and built classifier to bucket Lean programs by errors into eight different classes, and used it to classify 23,000 rollouts of model checkpoints
- Discovered that the model would drop certain open statements in 94% of ProofNet rollouts with open statements and traced the source from the training data and tested a SFT intervention, dropping prevalence of bad behavior to 0.8% of rollouts and improving pass@8 by 2.7 times on problems with open statements
- Built two Lean metaprograms to deploy research findings (data normalization tasks) across whole data corpus, integrating the metaprograms into the production data processing system built on Dagster
- Designed and implemented a synthetic-data generating Lean metaprogram to help teach mathlib API usage, improving correct usage of some withheld lemmas from 10.3% to 24.3% after SFT
- Designed experiments and evaluations to test and compare the efficacy of various premise selection tools on different benchmarks, and investigated why certain tools worked better for certain models

Software Engineering Intern | Capital One

June 2025 – August 2025

Python, Llama, Langchain

- Developed a pipeline around a model to automatically generate & edit mappings between cybersecurity frameworks (work generally done manually by analysts), scoring an 85% on a benchmark test
- Implemented various tools including MCP (model context protocol) server to call tools that implemented various semantic and syntax searching methods including RAG (retrieval augmented generation) and BM25

Undergrad Research | Carnegie Mellon University

August 2025 – May 2026

- Implemented a compiler optimization phase (defunctionalization) in the Resource Aware ML 2.0 research compiler under Jan Hoffmann
- Designed a new intermediate representation to facilitate defunctionalization through Lambda Set Specialization
- Semantically analyzed the code using a pipeline consisting of unification/constraint based analysis, collecting data on higher-order-function use

PROJECTS

Statically Typed Programming Language Compiler | OCaml, Menhir

- Built a compiler for a functional, higher-order typed ML-style programming language (based on Adjoint Natural Deduction) with standard functional programming features (algebraic datatypes, equirecursive types, nested pattern matching, bidirectional typechecking, mode inference)
- Types are tagged with a "mode" from a preorder of modes (substructural logics), representing a memory restriction on how they can be used (Linear, Affine, Strict, Structural), with the type checker checking that the conditions on values are satisfied using an additive approach
- Language compiles to an imperative IR called Adjoint SAX, enabling several memory optimizations (cut-id, reuse) and closure conversion, before being compiled into C with automated compile-time deallocation for linear and strict modes

OPEN SOURCE

Lean Open Source Contributions

- Formalized the Stone Space of types, proved properties (compactness + total separation) about it, and formalized important subsets and lemmas about the space and models in Lean
- Opened two PRs to Mathlib ([PR #32546](#) merged, [PR #33478](#) WIP)

TECHNICAL SKILLS

Relevant Skills: Lean, OCaml, Python, C, Java, SML, Agda